← Danh sách

0/30 câu đã làm · 0/28 câu chấm máy đúng lần đầu

Làm lại từ đầu

CĐ-08 · SQL cho phân tích dữ liệu

Luyện kiến thức: không tính giờ, chọn xong là biết đúng/sai và xem giải thích. Tiến độ lưu riêng với Luyện đề.

Mã đề **PETER-CD08-5615** · Người làm: Peter: gửi mã này cho mentor khi chữa bài.

Phần A — Nhận biết

Câu 1 · T08-A06 · 3 điểm · Trắc nghiệm · Nhận biết

Từ bảng `orders(id, customer_id, amount, ...)`, bạn cần danh sách các khách có **tổng** `amount` lớn hơn 250. Điều kiện `SUM(amount) > 250` phải đặt ở đâu?

- ○ **A.** Trong `WHERE`, vì `WHERE` là nơi đặt mọi điều kiện lọc
- ○ **B.** Đặt `SUM(amount) AS tong` ở SELECT rồi viết `WHERE tong > 250`
- ○ **C.** Trong `ORDER BY`, rồi dùng `LIMIT` để lấy các khách đứng đầu
- ○ **D.** Trong `HAVING` sau `GROUP BY customer_id`

Câu 2 · T08-A08 · 3 điểm · Trắc nghiệm · Nhận biết

Trong PostgreSQL, đặt `EXPLAIN` và `EXPLAIN ANALYZE` trước cùng một câu lệnh thì khác nhau chủ yếu ở điểm nào?

- ○ **A.** `EXPLAIN ANALYZE` chạy thật câu lệnh, in thêm thời gian và số dòng thực tế
- ○ **B.** `EXPLAIN ANALYZE` chạy câu lệnh ở chế độ chỉ đọc nên an toàn với `DELETE`
- ○ **C.** Hai lệnh như nhau, `ANALYZE` chỉ in thêm chi phí ước tính chi tiết hơn
- ○ **D.** `EXPLAIN ANALYZE` cập nhật thống kê bảng như lệnh `ANALYZE`, không chạy câu lệnh

Câu 3 · T08-A02 · 3 điểm · Trắc nghiệm · Nhận biết

Một câu truy vấn có đủ các mệnh đề `SELECT`, `FROM`, `WHERE`, `GROUP BY`, `HAVING`, `ORDER BY`. Theo **thứ tự xử lý logic** (thứ tự định nghĩa kết quả, không phải thứ tự viết), dãy nào đúng?

- ○ **A.** FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY
- ○ **B.** FROM → WHERE → SELECT → GROUP BY → HAVING → ORDER BY
- ○ **C.** SELECT → FROM → WHERE → GROUP BY → HAVING → ORDER BY
- ○ **D.** FROM → GROUP BY → WHERE → HAVING → SELECT → ORDER BY

Câu 4 · T08-A04 · 3 điểm · Trắc nghiệm · Nhận biết · tình huống

Shop thời trang muốn sinh mọi biến thể sản phẩm. Bảng `sizes` có 3 dòng (S, M, L), bảng `colors` có 4 dòng (do, xanh, den, trang). Truy vấn sau trả về số nào?

```
SELECT COUNT(*) FROM sizes CROSS JOIN colors;
```

- ○ **A.** 7
- ○ **B.** 4
- ○ **C.** 12
- ○ **D.** 3

Câu 5 · T08-A03 · 3 điểm · Trắc nghiệm · Nhận biết

Bảng `customers` có 5 khách (id 1 đến 5); bảng `orders` chỉ có đơn của khách 1, 2, 3. Bạn muốn kết quả **giữ đủ 5 khách**, khách chưa có đơn thì các cột lấy từ `orders` là NULL. Điền phép nối vào chỗ trống:

```
SELECT c.name, o.id
FROM customers c _____ orders o ON o.customer_id = c.id;
```

- ○ **A.** `INNER JOIN`
- ○ **B.** `CROSS JOIN`
- ○ **C.** `RIGHT JOIN`

- **D.** `LEFT JOIN`

Câu 6 · T08-A07 · 3 điểm · Trắc nghiệm · Nhận biết

Bảng `orders` có 7 dòng thuộc 3 khách (`customer_id` 1, 2, 3); một dòng có `amount` là NULL. Truy vấn sau trả về bao nhiêu dòng?

```
SELECT id, customer_id,
       SUM(amount) OVER (PARTITION BY customer_id) AS tong_khach
FROM orders;
```

- **A.** 1
- **B.** 7
- **C.** 3
- **D.** 6

Câu 7 · T08-A05 · 3 điểm · Trắc nghiệm · Nhận biết

Cột `amount` của bảng `orders` có 7 dòng với các giá trị: 200, 150, 300, 120, 80, NULL, 150. Biểu thức nào trả về **số đơn có ghi số tiền** (tức là 6)?

- **A.** `COUNT(amount)`
- **B.** `COUNT(*)`
- **C.** `COUNT(DISTINCT amount)`
- **D.** `COUNT(amount IS NOT NULL)`

Câu 8 · T08-A01 · 3 điểm · Trắc nghiệm · Nhận biết

Bảng `customers`:

id	name	city
1	An	HN
2	Binh	HCM
3	Chi	HN

4	Dung	DN
5	Ha	NULL

Khách `Ha` chưa khai báo thành phố nên `city` là NULL. Điều kiện `WHERE` nào lọc ra **đúng những khách chưa có thành phố**?

- ○ **A.** `WHERE city IN (NULL)`
- ○ **B.** `WHERE city = ''`
- ○ **C.** `WHERE city = NULL`
- ○ **D.** `WHERE city IS NULL`

Phần B — Thông hiểu

Câu 9 · T08-B01 · 3,5 điểm · Điền kết quả · Thông hiểu

Bảng `orders` (`order_date` lưu dạng chuỗi `'YYYY-MM-DD'`):

id	customer_id	order_date	amount	status
101	1	2026-01-05	200	paid
102	1	2026-01-20	150	paid
103	2	2026-01-11	300	cancelled
104	3	2026-02-02	120	paid
105	2	2026-02-15	80	paid
106	1	2026-02-28	NULL	pending
107	3	2026-03-03	150	paid

Truy vấn sau trả về gì? **Ghi mỗi dòng một bản ghi theo đúng thứ tự trả về, các cột cách nhau bằng dấu `|` (không thêm khoảng trắng), không ghi tên cột.**

```
SELECT id
FROM orders
WHERE status = 'pending' OR status = 'paid' AND amount >= 150
ORDER BY id;
```

Gõ đúng kết quả in ra, mỗi dòng một dòng…

Kiểm tra

Câu 10 · T08-B06 · 3,5 điểm · Điền kết quả · Thông hiểu

Bảng `orders` (`order_date` lưu dạng chuỗi `'YYYY-MM-DD'`):

id	customer_id	order_date	amount	status
101	1	2026-01-05	200	paid
102	1	2026-01-20	150	paid
103	2	2026-01-11	300	cancelled
104	3	2026-02-02	120	paid
105	2	2026-02-15	80	paid
106	1	2026-02-28	NULL	pending
107	3	2026-03-03	150	paid

Truy vấn sau trả về gì? **Ghi mỗi dòng một bản ghi theo đúng thứ tự trả về, các cột cách nhau bằng dấu `|` (không thêm khoảng trắng), không ghi tên cột.**

```
SELECT customer_id, COUNT(*), COUNT(amount), SUM(amount)
FROM orders
GROUP BY customer_id
ORDER BY customer_id;
```

Gõ đúng kết quả in ra, mỗi dòng một dòng…

Kiểm tra

Câu 11 · T08-B10 · 3 điểm · Trắc nghiệm · Thông hiểu · tình huống

Đội marketing loại khách trong danh sách đen trước khi gửi khuyến mãi. Bảng `customers` có id 1, 2, 3, 4, 5. Bảng `blacklist(customer_id)` có 2 dòng: 4 và NULL (dòng NULL do import file lỗi). Truy vấn sau trả về số nào?

```
SELECT COUNT(*)
FROM customers
WHERE id NOT IN (SELECT customer_id FROM blacklist);
```

- ○ **A.** 4
- ○ **B.** 5
- ○ **C.** 1
- ○ **D.** 0

Câu 12 · T08-B07 · 3 điểm · Trắc nghiệm · Thông hiểu

Bảng `customers(id, name, city)` và `orders(id, customer_id, ...)`; thành phố HN có hai khách đã mua (An, Chi). Câu truy vấn sau chạy trên PostgreSQL và trên SQLite thì thế nào?

```
SELECT c.city, c.name, COUNT(*) AS so_don
FROM customers c
JOIN orders o ON o.customer_id = c.id
GROUP BY c.city;
```

- ○ **A.** PostgreSQL chạy được và lấy name nhỏ nhất theo chữ cái trong mỗi nhóm
- ○ **B.** PostgreSQL báo lỗi vì `c.name` không có trong GROUP BY hay hàm gộp; SQLite vẫn chạy
- ○ **C.** Cả hai trả mỗi cặp (city, name) một dòng vì GROUP BY tự thêm cột thiếu
- ○ **D.** Cả hai đều báo lỗi vì không được dùng `COUNT(*)` sau khi JOIN hai bảng

Câu 13 · T08-B04 · 3,5 điểm · Điền kết quả · Thông hiểu · tình huống

Đội CRM cần danh sách khách **chưa từng đặt đơn** để gửi mã chào mừng.

Bảng `customers`:

id	name	city
1	An	HN
2	Binh	HCM
3	Chi	HN
4	Dung	DN
5	Ha	NULL

Bảng `orders` có 7 đơn (cột khoá `id` không bao giờ NULL) với `customer_id` lần lượt là 1, 1, 2, 3, 2, 1, 3. Truy vấn sau trả về gì? **Ghi mỗi dòng một bản ghi theo đúng thứ tự trả về, các cột cách**

nhau bằng dấu `|` (không thêm khoảng trắng), không ghi tên cột.

```
SELECT c.name
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.id
WHERE o.id IS NULL
ORDER BY c.id;
```

Gõ đúng kết quả in ra, mỗi dòng một dòng…

Kiểm tra

Câu 14 · T08-B03 · 3 điểm · Trắc nghiệm · Thông hiểu · tình huống

Analyst muốn liệt kê **cả 5 khách** kèm số đơn `paid` của từng người (khách chưa có đơn paid thì 0).

Bảng `customers`:

id	name	city
1	An	HN
2	Binh	HCM
3	Chi	HN
4	Dung	DN
5	Ha	NULL

Bảng `orders` (`order_date` lưu dạng chuỗi `'YYYY-MM-DD'`):

id	customer_id	order_date	amount	status
101	1	2026-01-05	200	paid
102	1	2026-01-20	150	paid
103	2	2026-01-11	300	cancelled
104	3	2026-02-02	120	paid
105	2	2026-02-15	80	paid
106	1	2026-02-28	NULL	pending
107	3	2026-03-03	150	paid

Câu truy vấn đã viết:

```
SELECT c.name, COUNT(o.id) AS so_don
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.id
WHERE o.status = 'paid'
GROUP BY c.id, c.name
ORDER BY c.id;
```

Kết quả có mấy dòng, và sửa thế nào cho đúng yêu cầu?

- ○ **A.** 3 dòng; chỉ cần đổi `COUNT(o.id)` thành `COUNT(*)`
- ○ **B.** 5 dòng, Dung và Ha có `so_don` = 0; truy vấn đã đúng
- ○ **C.** 3 dòng; chuyển `o.status = 'paid'` từ WHERE vào điều kiện ON
- ○ **D.** 3 dòng; đổi `LEFT JOIN` thành `RIGHT JOIN` để giữ khách chưa mua

Câu 15 · T08-B08 · 3 điểm · Trắc nghiệm · Thông hiểu

Bảng `scores(name, score)` có 4 dòng: (Lan, 90), (Minh, 85), (Nga, 85), (Oanh, 80). Với dòng của **Oanh**, ba cột `rn`, `rk`, `drk` nhận giá trị nào?

```
SELECT name,
       ROW_NUMBER() OVER (ORDER BY score DESC) AS rn,
       RANK()       OVER (ORDER BY score DESC) AS rk,
       DENSE_RANK() OVER (ORDER BY score DESC) AS drk
FROM scores;
```

- ○ **A.** rn = 4, rk = 3, drk = 3
- ○ **B.** rn = 4, rk = 4, drk = 4
- ○ **C.** rn = 4, rk = 4, drk = 3
- ○ **D.** rn = 3, rk = 4, drk = 3

Câu 16 · T08-B02 · 3 điểm · Trắc nghiệm · Thông hiểu · tình huống

Bảng `tickets` của bộ phận chăm sóc khách hàng có cột `status` gồm 6 dòng: `'open'`, `'closed'`, `'closed'`, NULL, `'open'`, NULL (NULL là vé mới tạo, chưa phân loại). Trưởng nhóm muốn đếm mọi vé **chưa đóng**, kể cả vé chưa phân loại, nên đáp số đúng là 4. Với `SELECT COUNT(*) FROM tickets WHERE ...`, điều kiện nào cho đúng 4?

- A. `status <> 'closed' OR status IS NULL`
- B. `NOT (status = 'closed')`
- C. `status <> 'closed' AND status IS NOT NULL`
- D. `status <> 'closed'`

Câu 17 · T08-B09 · 3,5 điểm · Điền kết quả · Thông hiểu · tình huống

Bảng `daily_sales` của một cửa hàng (đơn vị: triệu đồng):

day	revenue
2026-03-01	100
2026-03-02	130
2026-03-03	90
2026-03-04	90

Truy vấn tính chênh lệch so với ngày trước trả về gì? **Ghi mỗi dòng một bản ghi theo đúng thứ tự trả về, các cột cách nhau bằng dấu `|` (không thêm khoảng trắng), không ghi tên cột.**

```
SELECT revenue - LAG(revenue, 1, 0) OVER (ORDER BY day) AS chenh
FROM daily_sales
ORDER BY day;
```

Gõ đúng kết quả in ra, mỗi dòng một dòng…

Kiểm tra

Câu 18 · T08-B05 · 3 điểm · Trắc nghiệm · Thông hiểu · tình huống

Dashboard doanh thu chạy truy vấn sau cho tháng 4/2026 (`order_date` lưu dạng chuỗi `'YYYY-MM-DD'`). Bảng `orders` hiện **chưa có đơn nào** trong tháng 4.

```
SELECT COUNT(*) AS so_don, SUM(amount) AS doanh_thu
FROM orders
WHERE order_date >= '2026-04-01' AND order_date < '2026-05-01';
```

Kết quả là gì (trên cả PostgreSQL và SQLite)?

- ○ **A.** Một dòng: `so_don` = NULL, `doanh_thu` = NULL
- ○ **B.** Một dòng: `so_don` = 0, `doanh_thu` = 0
- ○ **C.** Không có dòng nào (tập kết quả rỗng)
- ○ **D.** Một dòng: `so_don` = 0, `doanh_thu` = NULL

Phần C — Vận dụng

Câu 19 · T08-C06 · 3 điểm · Trắc nghiệm · Vận dụng · tình huống

Bạn cần lấy **đơn có amount lớn nhất của từng khách** (mỗi khách một dòng, gồm id đơn và amount) từ bảng `orders(id, customer_id, amount, ...)`; cột amount có thể NULL. Truy vấn phải chạy được trên cả PostgreSQL và SQLite. Cách nào đúng?

- ○ **A.** Tính `rn` bằng ROW_NUMBER trong SELECT rồi lọc ngay ở cùng tầng truy vấn bằng mệnh đề `QUALIFY rn = 1`
- ○ **B.** Đặt window function thẳng vào điều kiện lọc: `WHERE ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY amount DESC) = 1`
- ○ **C.** Tính `ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY amount DESC) AS rn` trong một CTE (lọc `amount IS NOT NULL`), rồi ở truy vấn ngoài lọc `WHERE rn = 1`
- ○ **D.** Viết `GROUP BY customer_id` kèm `HAVING amount = MAX(amount)` để chỉ giữ dòng có amount lớn nhất của mỗi khách

Câu 20 · T08-C08 · 3 điểm · Trắc nghiệm · Vận dụng · tình huống

Trong PostgreSQL, bảng `orders` có 5 triệu dòng của khoảng 200 nghìn khách khác nhau, và chỉ có một index B-tree:

```
CREATE INDEX idx_cust_date ON orders (customer_id, order_date);
```

Với `SELECT * FROM orders ...`, điều kiện nào **khó tận dụng index này nhất** (EXPLAIN nhiều khả năng cho `Seq Scan`), biết rằng khoảng một phần ba số đơn có `order_date` từ 2026 trở đi?

- ○ **A.** `WHERE customer_id IN (42, 77)`
- ○ **B.** `WHERE customer_id = 42 AND order_date >= '2026-01-01'`

○ **C.** `WHERE customer_id = 42 ORDER BY order_date DESC LIMIT 5`

○ **D.** `WHERE order_date >= '2026-01-01'`

Câu 21 · T08-C09 · 6 điểm · Code viết tay · Vận dụng · tình huống

Ban giám đốc cần **báo cáo theo thành phố** từ hai bảng sau.

Bảng `customers`:

id	name	city
1	An	HN
2	Binh	HCM
3	Chi	HN
4	Dung	DN
5	Ha	NULL

Bảng `orders` (`order_date` lưu dạng chuỗi `'YYYY-MM-DD'`):

id	customer_id	order_date	amount	status
101	1	2026-01-05	200	paid
102	1	2026-01-20	150	paid
103	2	2026-01-11	300	cancelled
104	3	2026-02-02	120	paid
105	2	2026-02-15	80	paid
106	1	2026-02-28	NULL	pending
107	3	2026-03-03	150	paid

Viết một truy vấn SQL (chạy được trên cả PostgreSQL và SQLite) trả về 4 cột:

- `thanh_pho`: thành phố của khách; khách có `city` NULL ghi là `'Chua ro'`.
- `so_khach`: số khách thuộc thành phố đó (kể cả khách chưa mua).
- `so_khach_mua`: số khách có **ít nhất một** đơn `paid`.
- `doanh_thu`: tổng `amount` của các đơn `paid`; thành phố không có đơn paid ghi **0** (không để NULL).

Mọi thành phố đều phải xuất hiện. Sắp xếp theo `doanh_thu` giảm dần, hoà thì theo `thanh_pho` tăng dần. Với dữ liệu trên, kết quả đúng là:

thanh_pho	so_khach	so_khach_mua	doanh_thu

HN	2	2	620
HCM	1	1	80
Chua ro	1	0	0
DN	1	0	0

Truy vấn phải đúng cả khi dữ liệu thay đổi (không viết cứng tên thành phố hay con số).

```
Viết code Python như trên giấy (không chạy được)…
```

Xem bài mẫu & rubric Viết thử trước rồi hãy mở.

Câu 22 · T08-C03 · 3 điểm · Trắc nghiệm · Vận dụng · tình huống

Shop phân hạng đơn theo giá trị bằng bảng `tiers`:

tier	lo	hi
Dong	0	99
Bac	100	200
Vang	200	1000

Bảng `orders` có 7 đơn với `amount` lần lượt là 200, 150, 300, 120, 80, NULL, 150. Truy vấn sau cho kết quả nào?

```
SELECT t.tier, COUNT(o.id) AS so_don
FROM tiers t
LEFT JOIN orders o ON o.amount BETWEEN t.lo AND t.hi
GROUP BY t.tier, t.lo
ORDER BY t.lo;
```

- ○ **A.** Dong 1, Bac 4, Vang 2
- ○ **B.** Dong 2, Bac 4, Vang 2
- ○ **C.** Dong 1, Bac 4, Vang 2, thêm một dòng tier NULL cho đơn amount NULL
- ○ **D.** Dong 1, Bac 3, Vang 2

Câu 23 · T08-C04 · 3 điểm · Trắc nghiệm · Vận dụng · tình huống

Bảng `orders` có 7 đơn, trong đó đúng 1 đơn có `status = 'cancelled'` (cột `amount` có một giá trị NULL ở đơn khác). Analyst tính tỉ lệ huỷ:

```
SELECT SUM(CASE WHEN status = 'cancelled' THEN 1 ELSE 0 END) / COUNT(*) AS ti_le_huy
FROM orders;
```

Kết quả `ti_le_huy` trên cả PostgreSQL và SQLite là gì?

- ○ **A.** Khoảng 0,1429 (tức là 1/7)
- ○ **B.** NULL, vì cột amount có một giá trị NULL
- ○ **C.** 0, vì hai vế là số nguyên nên phép chia bỏ phần lẻ
- ○ **D.** 1, vì SUM đếm được đúng 1 đơn bị huỷ

Câu 24 · T08-C01 · 3 điểm · Trắc nghiệm · Vận dụng · tình huống

Kế toán đối soát doanh thu và tiền đã thu. Đơn 101 (amount 200) được khách trả **hai lần**, đơn 102 (amount 150) trả một lần:

orders.id	amount
101	200
102	150

payments.order_id	paid
101	120
101	80
102	150

```
SELECT SUM(o.amount) AS doanh_thu, SUM(p.paid) AS da_thu
FROM orders o
JOIN payments p ON p.order_id = o.id
WHERE o.id IN (101, 102);
```

Hai cột `doanh_thu` và `da_thu` lần lượt bằng bao nhiêu?

- ○ **A.** 550 và 550
- ○ **B.** 350 và 270

- **C.** 350 và 350
- **D.** 550 và 350

Câu 25 · T08-C07 · 3,5 điểm · Điền kết quả · Vận dụng · tình huống

Chuỗi hai cửa hàng theo dõi doanh thu lũy kế (đơn vị: triệu đồng). Bảng `daily_sales`:

store	day	revenue
A	2026-03-01	100
A	2026-03-02	130
A	2026-03-03	90
B	2026-03-01	200
B	2026-03-02	150

Truy vấn sau trả về gì? **Ghi mỗi dòng một bản ghi theo đúng thứ tự trả về, các cột cách nhau bằng dấu `|` (không thêm khoảng trắng), không ghi tên cột.**

```
SELECT store,
       SUM(revenue) OVER (PARTITION BY store ORDER BY day) AS luy_ke
FROM daily_sales
ORDER BY store, day;
```

Gõ đúng kết quả in ra, mỗi dòng một dòng…

Kiểm tra

Câu 26 · T08-C02 · 3,5 điểm · Điền kết quả · Vận dụng · tình huống

Phòng nhân sự cần bảng "nhân viên, quản lý trực tiếp" từ bảng `employees`:

id	name	manager_id
1	Lan	NULL
2	Minh	1
3	Nga	1
4	Oanh	2

Truy vấn sau trả về gì? **Ghi mỗi dòng một bản ghi theo đúng thứ tự trả về, các cột cách nhau bằng dấu `|` (không thêm khoảng trắng), không ghi tên cột.**

```
SELECT e.name, COALESCE(m.name, '-') AS manager
FROM employees e
LEFT JOIN employees m ON e.manager_id = m.id
ORDER BY e.id;
```

Gõ đúng kết quả in ra, mỗi dòng một dòng…

Kiểm tra

Câu 27 · T08-C05 · 3,5 điểm · Điền kết quả · Vận dụng · tình huống

Đội bán hàng tìm khách **trung thành**: có từ 2 đơn `paid` trở lên, xếp theo tổng tiền giảm dần.

Bảng `orders` (`order_date` lưu dạng chuỗi `'YYYY-MM-DD'`):

id	customer_id	order_date	amount	status
101	1	2026-01-05	200	paid
102	1	2026-01-20	150	paid
103	2	2026-01-11	300	cancelled
104	3	2026-02-02	120	paid
105	2	2026-02-15	80	paid
106	1	2026-02-28	NULL	pending
107	3	2026-03-03	150	paid

Truy vấn sau trả về gì? **Ghi mỗi dòng một bản ghi theo đúng thứ tự trả về, các cột cách nhau bằng dấu `|` (không thêm khoảng trắng), không ghi tên cột.**

```
SELECT customer_id, COUNT(*) AS so_don, SUM(amount) AS tong
FROM orders
WHERE status = 'paid'
GROUP BY customer_id
HAVING COUNT(*) >= 2
ORDER BY tong DESC;
```

Gõ đúng kết quả in ra, mỗi dòng một dòng…

Kiểm tra

Phần D — Vận dụng cao

Câu 28 · T08-D01 · 3 điểm · Trắc nghiệm · Vận dụng cao · tình huống

Ví điện tử vẽ biểu đồ tiền nạp lũy kế **theo từng giao dịch**. Bảng `payments_log`:

id	day	amount
1	2026-03-01	100
2	2026-03-02	50
3	2026-03-02	70
4	2026-03-03	30

```
SELECT id, day, amount,
       SUM(amount) OVER (ORDER BY day) AS luy_ke
FROM payments_log
ORDER BY id;
```

Cột `luy_ke` (theo thứ tự id 1, 2, 3, 4) là gì, và sửa thế nào để mỗi giao dịch có một giá trị lũy kế riêng?

- ○ **A.** 250, 250, 250, 250; cần thêm `PARTITION BY day` vào OVER
- ○ **B.** 100, 220, 220, 250; sửa bằng cách thêm `PARTITION BY day` vào OVER
- ○ **C.** 100, 220, 220, 250; thêm `id` vào sau `day` trong ORDER BY của OVER
- ○ **D.** 100, 150, 220, 250; truy vấn đã cộng dồn từng dòng, không cần sửa

Câu 29 · T08-D02 · 3,5 điểm · Điền kết quả · Vận dụng cao · tình huống

Bộ phận chống gian lận tìm các đơn **lớn hơn mức trung bình của chính khách đó**.

Bảng `orders` (`order_date` lưu dạng chuỗi `'YYYY-MM-DD'`):

id	customer_id	order_date	amount	status
101	1	2026-01-05	200	paid
102	1	2026-01-20	150	paid
103	2	2026-01-11	300	cancelled

104	3	2026-02-02	120	paid
105	2	2026-02-15	80	paid
106	1	2026-02-28	NULL	pending
107	3	2026-03-03	150	paid

Truy vấn sau trả về gì? **Ghi mỗi dòng một bản ghi theo đúng thứ tự trả về, các cột cách nhau bằng dấu `|` (không thêm khoảng trắng), không ghi tên cột.**

```
SELECT o.id, o.amount
FROM orders o
WHERE o.amount > (
  SELECT AVG(o2.amount)
  FROM orders o2
  WHERE o2.customer_id = o.customer_id
)
ORDER BY o.id;
```

Gõ đúng kết quả in ra, mỗi dòng một dòng…

Kiểm tra

Câu 30 · T08-D03 · 6 điểm · Code viết tay · Vận dụng cao · tình huống

Bạn làm báo cáo tháng cho shop online.

Bảng `orders` (`order_date` lưu dạng chuỗi `'YYYY-MM-DD'`):

id	customer_id	order_date	amount	status
101	1	2026-01-05	200	paid
102	1	2026-01-20	150	paid
103	2	2026-01-11	300	cancelled
104	3	2026-02-02	120	paid
105	2	2026-02-15	80	paid
106	1	2026-02-28	NULL	pending
107	3	2026-03-03	150	paid

Viết một truy vấn SQL dùng CTE (`WITH`) và window function, chạy được trên cả PostgreSQL và SQLite, trả về mỗi tháng một dòng gồm:

- `thang`: chuỗi `'YYYY-MM'`, lấy bằng `SUBSTR(order_date, 1, 7)`.
- `doanh_thu`: tổng `amount` của các đơn `paid` trong tháng.
- `tang_truong_pct`: phần trăm thay đổi so với tháng liền trước, $100 \cdot \frac{\text{doanh thu} - \text{doanh thu tháng trước}}{\text{doanh thu tháng trước}}$, làm tròn 1 chữ số thập phân; tháng đầu tiên là NULL; nếu tháng trước bằng 0 thì cũng trả NULL thay vì báo lỗi.
- `luy_ke`: doanh thu cộng dồn từ tháng đầu tiên tới tháng đó.

Sắp xếp theo `thang` tăng dần. Không dùng self-join. Với dữ liệu trên, kết quả đúng là:

thang	doanh_thu	tang_truong_pct	luy_ke
2026-01	350	NULL	350
2026-02	200	-42.9	550
2026-03	150	-25.0	700

Viết code Python như trên giấy (không chạy được)…

Xem bài mẫu & rubric Viết thử trước rồi hãy mở.